

Incremental SMT Verification for LLM Code Repair: A Failed Validation and Dataset Contribution

Anonymous Authors¹

Abstract

This paper reports on a validation attempt for the hypothesis that incremental SMT verification would reduce verification time by 2–5× compared to batch re-verification in iterative LLM code repair workflows for typed Python. The validation failed due to infrastructure errors: all 48 LLM code generation attempts encountered HTTP 401 authentication failures, yielding a 0% constraint extraction rate and preventing empirical hypothesis testing. However, the dataset extension pipeline succeeded, producing 100 mechanically-generated Pydantic-annotated HumanEval prompts. The constraint extraction pipeline architecture was validated on error cases. The hypothesis remains theoretically plausible but empirically unvalidated. This work contributes (1) a reusable typed benchmark dataset (HumanEval + Pydantic extensions), (2) a modular constraint extraction pipeline ready for retry, and (3) analysis of experimental failure modes demonstrating that infrastructure pre-validation is critical for experiments with external API dependencies.

1. Introduction

Large language models have demonstrated code generation capabilities on benchmarks such as HumanEval and APPS, yet they lack formal correctness guarantees. Test suites detect obvious errors but miss edge cases that static analysis and satisfiability modulo theories (SMT) solvers can identify. However, batch SMT verification does not scale well to iterative debugging workflows: re-verifying entire programs at each repair

step becomes computationally prohibitive.

Prior SMT-guided program repair systems such as Angelix and Prophet target small human-written patches, not full LLM-generated programs. Neural code generation systems such as AlphaCode and CodeT5 rely exclusively on test-suite validation, providing no formal verification. This gap motivates the hypothesis that incremental SMT verification—re-verifying only modified functions and their dependencies—could reduce verification time for LLM repair workflows in statically-typed languages.

The hypothesis builds on established components: static analyzers (Pyre for Python, Prusti for Rust) extract SMT constraints from type annotations in human-written code, Z3 supports incremental solving via push/pop contexts, and neural repair literature indicates that 80% of human code repairs touch 1–3 lines. However, transferability to LLM-generated code remained unverified.

This work designed a four-hypothesis validation chain: h-e1 (existence hypothesis: static analyzers can extract SMT constraints from LLM-generated typed code at $\geq 90\%$ success rate), followed by three mechanism hypotheses testing constraint extraction (h-m1), dependency analysis (h-m2), and speedup measurement (h-m3). The experimental protocol required extending HumanEval with Pydantic type annotations to create a typed benchmark.

The validation attempt encountered an invalid Anthropic API key, blocking all 48 LLM generation attempts with HTTP 401 errors. Without generated code, the constraint extraction pipeline could not test whether type annotations enable SMT predicate mapping. The hypothesis remains theoretically plausible but empirically unvalidated.

Despite generation failure, the dataset extension pipeline succeeded: 100 Pydantic-annotated prompts were mechanically generated from HumanEval. The constraint extraction architecture (DatasetExtender → LLMGenerator → PyreExtractor → Z3Validator → MetricsEngine) was validated on error files, confirming

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

that the pipeline processes malformed inputs without failure.

This paper contributes:

1. **HumanEval + Pydantic Type Extensions:** A mechanically-generated typed benchmark dataset of 100 prompts suitable for SMT constraint extraction research, addressing the gap in typed benchmarks for formal verification of neural code generation.
1. **Constraint Extraction Pipeline Architecture:** A modular five-component pipeline validated on negative cases and ready for retry with valid API credentials.

The primary methodological contribution is the demonstration that infrastructure reliability is a prerequisite for hypothesis validation. Experimental pipelines must address failure modes explicitly through pre-flight validation, fast-fail logic for non-retryable errors, and checkpoint/resume mechanisms, or risk perpetual blockage.

Section 2 positions this work against SMT-guided repair for human code and neural code generation. Section 3 describes the hypothesis decomposition, experimental design, and pipeline architecture. Section 4 presents results: dataset extension succeeded, LLM generation failed, hypothesis untested. Section 5 discusses implications and limitations. Section 6 concludes with recommendations for retry and future work.

2. Related Work

This work builds on SMT-guided program repair, neural code generation, and static analysis for typed languages. The hypothesis—incremental SMT for LLM code repair in typed Python—lies at their intersection, though empirical validation was blocked.

2.0.1. SMT-GUIDED PROGRAM REPAIR

Angelix and Prophet apply SMT solvers to synthesize patches for buggy programs by encoding correctness specifications as constraints. These systems work on small human-written patches (1–10 lines) and assume localized modifications. Constraint extraction relies on predefined templates for common bug patterns. While effective for human code repair, they do not scale to full LLM-generated programs where bug locations are unknown and programs exceed 100 lines.

The hypothesis proposed in this work extends this

paradigm to LLM code repair workflows: instead of batch re-verification on each iteration, incremental SMT re-verifies only modified functions and dependencies. The key differences are scale (full programs, not patches) and code source (LLM-generated code may have different quality characteristics). However, transferability remains untested due to validation failure.

2.0.2. NEURAL CODE GENERATION

AlphaCode and CodeT5 demonstrate that large-scale pretraining enables LLMs to generate functionally correct code on benchmarks. These systems rely on test-suite validation: generate multiple candidates, filter by passing tests, rank by confidence. While effective for programming competitions, test suites provide incomplete correctness guarantees.

Formal verification could address this gap by providing SMT-based correctness proofs. However, existing neural code generation systems do not integrate static analysis or SMT solvers. This work proposed adding incremental SMT verification to LLM repair loops, but could not validate whether LLM-generated code has extractable constraints or whether SMT provides practical benefit over test suites.

2.0.3. STATIC ANALYSIS FOR TYPED LANGUAGES

Prusti (Rust) and Pyre (Python) extract SMT constraints from type annotations and function contracts. These tools demonstrate that type annotations map to first-order logic predicates suitable for SMT solving. Prusti uses Rust ownership types and developer-written specifications; Pyre uses Python type hints and optional runtime assertions.

Prior work validates these analyzers on human-written typed code. Whether LLM-generated code has sufficient annotation quality for constraint extraction is an open question. This work hypothesized $\geq 90\%$ extraction success, but could not test this due to API authentication failures. The constraint extraction pipeline was validated on error files, confirming architectural soundness on negative cases, but the core claim remains empirically unverified.

2.0.4. NEURAL PROGRAM REPAIR

CURE and CoCoNut demonstrate that neural program repair typically modifies 1–3 lines in human code (80% of repairs are localized). The hypothesis assumed LLM repairs exhibit similar locality. If true, dependency cones remain small ($<30\%$), enabling incremental SMT speedup. However, LLM repair patterns may differ from human edits—broader modifications, cross-

module dependencies—which could invalidate speedup claims. Repair locality could not be tested due to prerequisite failures.

2.0.5. BENCHMARKS FOR TYPED CODE

HumanEval provides 164 Python programming problems with test suites but lacks type annotations required for static analysis. APPS and CodeContests similarly focus on functional correctness without formal specifications. This work fills this gap by mechanically generating 100 type-annotated prompts. This artifact is reusable for future constraint extraction research, independent of the validation failure.

2.0.6. POSITIONING

This hypothesis occupies unexplored territory: combining SMT-guided repair (established for human patches) with neural code generation (lacking verification) in typed Python with Pydantic. The key novelty is incremental SMT for full LLM programs, but this remains a proposal pending validation. The verified contribution is the dataset artifact and pipeline architecture.

3. Methodology

A four-hypothesis validation chain was designed to test whether incremental SMT verification would reduce verification time for LLM code repair in typed Python with Pydantic. This section describes the hypothesis decomposition, experimental protocol, and pipeline architecture. While validation was blocked by infrastructure failures, the methodology and implementation artifacts remain sound.

3.0.1. HYPOTHESIS DECOMPOSITION

Main Hypothesis (H-IncrementalSMT-v1): Under iterative LLM code repair workflows in typed Python with Pydantic, if incremental SMT verification is used (re-verifying only modified functions and dependencies), then verification time reduces compared to batch re-verification, because unchanged code portions skip redundant constraint checking.

This decomposes into four sub-hypotheses:

- **h-e1 (EXISTENCE):** Static analyzers can extract SMT constraints from LLM-generated typed code. Gate: $\geq 90\%$ extraction success rate. If fail: pivot to neural constraint extraction. **Status: FAIL (0.0%, infrastructure block).**
- **h-m1 (MECHANISM):** Pyre extracts type

contracts from Pydantic annotations. Gate: **MUST_WORK. Status: NOT_STARTED (prerequisite h-e1 failed).**

- **h-m2 (MECHANISM):** Dependency analysis computes transitive closure for incremental re-verification. Gate: **SHOULD_WORK. Status: NOT_STARTED.**
- **h-m3 (MECHANISM):** Incremental SMT achieves speedup vs batch. Gate: **MUST_WORK. Status: NOT_STARTED.**

The dependency chain is sequential: $h-e1 \rightarrow h-m1 \rightarrow h-m2 \rightarrow h-m3$. Gate failure at h-e1 blocked all downstream hypotheses.

3.0.2. DATASET EXTENSION: HUMANEVAL + PYDANTIC

HumanEval lacks type annotations required for SMT constraint extraction. A template-based transformation pipeline was designed:

Step 1: Parse function signature and docstring from each HumanEval problem.

Step 2: Generate Pydantic BaseModel schema for inputs and outputs:

```
class Input(BaseModel):
    param1: Type1
    param2: Type2

    @validator('param1')
    def validate_param1(cls, v):
        assert v is not None, 'param1 must not be None'
        return v
```

Step 3: Add @validator decorators encoding preconditions extracted from docstrings.

Step 4: Preserve original test suite for functional correctness validation.

This process was applied to HumanEval problems 0–99, generating 100 Pydantic-annotated prompts. All 100 prompts were created successfully, demonstrating feasibility of mechanical type annotation generation.

Design Rationale: Pydantic validators provide runtime-checkable contracts that map naturally to SMT predicates. Mypy type hints support static checking but lack explicit constraint semantics. HumanEval was chosen because it provides existing test suites and serves as a standard baseline for code generation research. Template-based transformation ensures consistent annotation quality and scales to large benchmarks,

though templates encode only simple constraint patterns (non-null, range checks). Complex preconditions require semantic understanding beyond pattern matching.

3.0.3. CONSTRAINT EXTRACTION PIPELINE

The pipeline consists of five sequential modules:

1. **DatasetExtender**: Load HumanEval, generate Pydantic templates (150 LOC).
2. **LLMGenerator**: Generate typed Python code via Claude Sonnet 3.5 API (82 LOC).
3. **PyreExtractor**: Parse AST, extract SMT constraints from type annotations (128 LOC).
4. **Z3Validator**: Validate constraint quality via SMT satisfiability checking (132 LOC).
5. **MetricsEngine**: Compute extraction/quality rates, generate visualizations (156 LOC).

Modular design allowed partial validation despite LLM generation failure. DatasetExtender was validated on 100 prompts (success). LLMGenerator failed with HTTP 401 errors (invalid API key). PyreExtractor was validated on error files (correctly found 0 constraints). Z3Validator was validated on error files (correctly identified 0 quality constraints). MetricsEngine correctly computed 0.0% extraction rate and generated 4 figures.

Pipeline validated on negative cases only. Pipeline separation (extend → generate → extract) prevented cascading failures—dataset extension succeeded independently of generation failure.

3.0.4. INCREMENTAL SMT STRATEGY

Batch Verification (Baseline): On each repair iteration, re-verify entire program by re-extracting all constraints and solving via Z3.

Incremental Verification (Proposed):

1. Extract constraints from initial program, solve via Z3 push/pop contexts.
2. On modification, run dependency analysis to compute invalidation cone (functions reachable from modified code).
3. Re-verify only invalidated constraints; reuse cached results for unchanged code.

Success Criteria: Not validated due to infrastructure failure.

Conservative dependency analysis was designed for soundness: over-approximate dependency cone to prioritize soundness. False positives (re-verifying more than necessary) reduce speedup but maintain correctness. False negatives (missing dependencies) cause unsound verification. Z3 incremental mode uses push/pop contexts to cache verification state across iterations. The approach assumes localized repairs (based on CURE/CoCoNut literature showing 80% of human repairs touch 1–3 lines), but this assumption remains unverified for LLM code.

3.0.5. VALIDATION PROTOCOL

h-e1 Protocol:

1. Generate 100 typed Python programs from Pydantic-annotated HumanEval prompts using Claude Sonnet 3.5.
2. Extract SMT constraints via Pyre static analyzer.
3. Measure extraction success rate (% of programs with usable constraints).
4. Compare against 90% threshold.

Actual Result: Step 1 failed with 100% API authentication errors. `extraction_rate = 0.0%` (infrastructure block, not extraction failure).

h-m1, h-m2, h-m3 protocols were not executed due to prerequisite h-e1 failure.

3.0.6. FAILURE MODE ANALYSIS

The h-e1 validation failed with 0.0% extraction rate due to 100% API authentication failure (48 generation attempts returned HTTP 401 "API key is invalid"). Root cause: invalid Anthropic API key in experiment configuration. The retry mechanism (3 attempts with exponential backoff) correctly exhausted retries but lacked logic to fast-fail on non-retryable auth errors.

The falsifier for h-e1 is "static analyzer fails to extract usable constraints (<90% success)." Infrastructure failures prevent testing the falsifier. The hypothesis remains theoretically plausible pending retry with valid API access. The pipeline is complete and validated on negative cases. Retry requires only fixing the API key, not redesigning the experiment.

3.0.7. REPRODUCIBILITY

Environment: Python 3.13, dependencies: `datasets`, `anthropic`, `pydantic`, `z3-solver`,

matplotlib.

Data: HumanEval (openai_humaneval, problems 0–99), extended prompts in src/data/humaneval_pydantic/.

Code: 700 LOC total, modular architecture.

Retry Instructions: Set valid ANTHROPIC_API_KEY, run python main.py. Expected outcome (if hypothesis valid): extraction_rate $\geq 90\%$, gate PASS.

4. Experimental Setup

The experimental design for validating the four sub-hypotheses was fully specified, though execution was blocked at h-e1.

4.0.1. RESEARCH QUESTIONS

RQ1 (Existence): Can static analyzers extract SMT constraints from LLM-generated typed code at $\geq 90\%$ success rate? (Untested due to infrastructure failure)

RQ2 (Mechanism): Does dependency analysis accurately identify invalidation cones while maintaining soundness? (Untested, prerequisite RQ1 failed)

RQ3 (Performance): Does incremental SMT achieve speedup vs batch re-verification on iterative LLM code repair? (Untested, prerequisite RQ1, RQ2 failed)

4.0.2. DATASET

Source: HumanEval (OpenAI), problems 0–99 (100 programs).

Extension: Mechanically generated Pydantic type annotations via template-based transformation. Each extended prompt includes function signature with type hints, Pydantic BaseModel schemas for inputs/outputs, @validator decorators encoding preconditions, and original docstring and test suite.

Rationale: HumanEval provides standard baseline for code generation research. Type extension enables static analysis without manual annotation bias. Programs span 10–50 LOC (suitable for proof-of-concept validation).

Validation: Dataset extension pipeline executed successfully, generating 100 JSON files in src/data/humaneval_pydantic/. Each file contains valid Pydantic template matching the original HumanEval problem specification.

4.0.3. LLM CODE GENERATION

Model: Claude Sonnet 3.5 (claude-sonnet-3-5-20240620)

Temperature: 0.2

Max Tokens: 512

System Prompt: "Generate typed Python code using Pydantic BaseModel. Include @validator decorators for preconditions. Do not use eval, exec, or metaprogramming."

Expected Output: 100 typed Python programs with Pydantic annotations, passing original HumanEval test suites.

Actual Result: 0 programs generated. All 48 generation attempts failed with HTTP 401 "API key is invalid" errors.

4.0.4. BASELINE METHODS

Baseline 1: Batch SMT Verification

- **Method:** Re-verify entire program on each repair iteration using Z3.
- **Correctness:** Sound and complete.
- **Use:** Primary performance baseline for h-m3 speedup measurement (not tested).

Baseline 2: Test-Suite-Only Validation

- **Method:** LLM generates code, run HumanEval test suite, accept if all tests pass.
- **Correctness:** Incomplete (misses edge cases not covered by tests).
- **Use:** Comparison point for verification overhead vs no verification (comparison not performed).

No baseline comparisons were conducted due to infrastructure failure.

4.0.5. EVALUATION METRICS

h-e1: Extraction Success Rate

$$\text{extraction_rate} = \frac{\text{programs with constraints}}{\text{total programs}} \times 100\%$$

h-m3: Speedup Factor (Not Measured)

$$\text{speedup} = \frac{\text{batch verification time}}{\text{incremental verification time}}$$

h-m2: Invalidation Cone Size (Not Measured)

$$\text{cone_size} = \frac{\text{constraints in invalidation cone}}{\text{total constraints}} \times 100\%$$

h-m2: Soundness (Error Detection Rate) (Not Measured)

$$\text{detection_rate} = \frac{\text{seeded errors caught}}{\text{total seeded errors}} \times 100\%$$

h-m1: Extraction Overhead (Not Measured)

$$\text{overhead_ratio} = \frac{\text{extraction time}}{\text{total verification time}} \times 100\%$$

4.0.6. THREATS TO VALIDITY

Internal Validity: Infrastructure failure (invalid API key) blocked testing of core hypothesis. Mitigated by isolating failure mode (authentication error, not hypothesis refutation).

External Validity: Single language (typed Python with Pydantic), limited to HumanEval-scale programs (10–50 LOC). Generalization to Rust + Prusti or larger codebases untested.

Construct Validity: Speedup measured as wall-clock time (includes extraction overhead, SMT solving, dependency analysis). Extraction overhead could dominate.

Conclusion Validity: No statistical tests conducted (no data).

5. Results

The validation attempt produced one artifact—HumanEval + Pydantic type extensions—while all hypothesis testing was blocked by infrastructure failures.

5.0.1. DATASET EXTENSION: ARTIFACT CONTRIBUTION

The dataset extension pipeline successfully generated 100 Pydantic-annotated prompts from HumanEval problems 0–99.

Manual inspection of 10 randomly-sampled prompts confirmed type annotations match original signatures, validators encode docstring preconditions, and test cases remain unchanged.

Artifact Availability: Dataset available in `src/data/humaneval\_pydantic/`. This artifact demonstrates that typed benchmarks can be mechanically generated from existing code generation datasets.

5.0.2. LLM GENERATION: INFRASTRUCTURE FAILURE

All LLM code generation attempts failed with HTTP 401 authentication errors.

Quantitative Results:

- Total generation attempts: 48
- Successful generations: 0 (0% success rate)
- Error type: HTTP 401 "API key is invalid" (100% of failures)
- Retry attempts per request: 3 (exponential back-off: 1s, 2s, 4s)

Error Message:

Error code: 401 - {'type': 'error', 'error': {'type': 'authentication_error', 'message': 'API key is invalid.'}}

Root Cause Analysis: Three competing explanations were evaluated. (1) Invalid API key: Error message explicitly states "API key is invalid"; consistent across all 48 attempts. Plausibility: HIGH. (2) API rate limiting: Would produce HTTP 429 errors, not 401. Plausibility: LOW. (3) Network/firewall blocking: Would produce connection timeouts, not authenticated 401 responses. Plausibility: LOW.

Most Likely Interpretation: Invalid API key. Retry with confirmed-valid key would distinguish between infrastructure failure and hypothesis refutation.

The h-e1 gate criterion ($\text{extraction_success_rate} \geq 90\%$) could not be tested. The hypothesis remains theoretically plausible but empirically unvalidated. All downstream hypotheses (h-m1, h-m2, h-m3) were blocked.

5.0.3. PIPELINE ARCHITECTURAL VALIDATION

Despite generation failure, the constraint extraction pipeline was validated on error files.

Architectural Soundness: Pipeline validated on negative cases only. PyreExtractor AST parser correctly handles unparseable Python, returns 0 constraints as expected. Z3Validator correctly identifies 0 quality constraints from empty input. MetricsEngine correctly computes 0% extraction rate and triggers FAIL condition.

The pipeline is architecturally sound and ready for retry. Fixing the API key requires no code changes—only environment configuration.

Component	Input	Expected Output	Actual Output	Status
DatasetExtender	HumanEval (100 problems)	100 Pydantic prompts	100 JSON files	PASS

Component	Input	Expected Output	Actual Output	Status
DatasetExtender	HumanEval (100 problems)	100 Pydantic prompts	100 JSON files	PASS
LLMGenerator	100 Pydantic prompts	100 typed programs	48 error files	FAIL (infrastructure)
PyreExtractor	48 error files	0 constraints	0 constraints	PASS (negative case)
Z3Validator	0 constraints	0 quality constraints	0 quality constraints	PASS (negative case)
MetricsEngine	0 constraints	extraction_rate=0%, gate FAIL	extraction_rate=0.0%, gate FAIL	PASS

5.0.4. VISUALIZATION RESULTS

Four figures were generated, though three contain placeholder/zero data.

Figure 1 (gate_metrics.png): Gate threshold (90.0%) vs actual extraction rate (0.0%). Shows gate failure.

Figure 2 (failure_modes.png): Categorization of extraction failures. 100% API authentication errors (48/48 attempts). This is the only figure with meaningful data.

Figure 3 (success_by_complexity.png): Planned analysis of extraction success by program complexity. All bins show 0% due to no successful generations.

Figure 2 is the critical evidence: the failure mode is homogeneous (100% authentication errors), not heterogeneous. This supports the interpretation that infrastructure failure, not scientific invalidity, blocked validation.

5.0.5. HYPOTHESIS VALIDATION SUMMARY

Assumptions:

- A1 (LLM code has extractable constraints): UNVERIFIED (h-e1 blocked)
- A2 (repair locality): UNVERIFIED (h-m2 not reached)
- A3 (dependency analysis soundness): UNVERIFIED (h-m2 not reached)
- A4 (extraction overhead small): UNVERIFIED (h-m1 not reached)
- A5 (no dynamic features): PARTIALLY VERIFIED (prompt constraints applied, not tested on actual LLM output)

6. Discussion

The validation attempt failed to test the hypothesis that incremental SMT verification would reduce verification time for LLM code repair in typed Python with Pydantic. This section interprets the results, acknowledges the validation blockage, and extracts lessons from the infrastructure failure.

6.0.1. KEY FINDINGS

Finding 1: Dataset Extension Is Feasible

The mechanically-generated HumanEval + Pydantic dataset demonstrates that typed benchmarks can be created from existing code generation datasets without manual annotation. Prior benchmarks (HumanEval, APPS, CodeContests) lack type annotations required for SMT constraint extraction. The template-based transformation approach (function signature + docstring → Pydantic BaseModel + validators) scales mechanically, generating 100 typed prompts in under 1 minute. However, templates encode simple constraint patterns (non-null, range checks). Complex preconditions require semantic understanding beyond pattern matching.

Finding 2: Infrastructure Failure Blocked Validation

All 48 LLM generation attempts failed with HTTP 401 authentication errors, preventing any testing of the core hypothesis. The failure mode is homogeneous (100% API auth errors) and external to the hypothesis (infrastructure misconfiguration, not scientific invalidity). The hypothesis remains theoretically plausible. The experimental pipeline is complete and validated on negative cases. Retry requires only fixing the API key—no redesign of experiment, implementation, or evaluation protocol.

Finding 3: Pipeline Architecture Prevented Cascading Failures

The modular design (DatasetExtender → LLMGenerator → PyreExtractor → Z3Validator → MetricsEn-

Hypothesis	Gate	Threshold	Actual Result	Status	Reason
h-e1	MUST_WORK	extraction_rate $\geq$ 90%	0.0%	FAIL	Infrastructure block (API auth)
h-m1	MUST_WORK	All programs extract constraints	N/A	NOT_STARTED	Prerequisite h-e1 failed
h-m2	SHOULD_WORK	cone_size <30%, detection_rate=100%	N/A	NOT_STARTED	Prerequisite h-e1, h-m1 failed
h-m3	MUST_WORK	median speedup $\geq 2x$	N/A	NOT_STARTED	Prerequisite h-e1, h-m1, h-m2 failed

gine) allowed partial validation despite generation failure. Dataset extension succeeded independently; constraint extraction components validated on error files. Tightly-coupled pipelines propagate failures. Modular separation enables isolating failure points and validating unaffected components.

6.0.2. LIMITATIONS AND BOUNDARY CONDITIONS

Limitation 1: Complete Validation Blockage—Infrastructure Failure

The entire hypothesis validation chain (h-e1 $\rightarrow$ h-m1 $\rightarrow$ h-m2 $\rightarrow$ h-m3) was blocked at the foundation due to LLM API authentication failures. Zero predictions were empirically tested. The hypothesis remains in a "proposed but untested" state. This is an infrastructure limitation, not hypothesis refutation. The hypothesis design is sound. The implementation artifacts are complete and validated on negative cases. Retry with valid API key would test the hypothesis.

Limitation 2: Unverified Assumption—LLM Code Quality for Static Analysis (A1)

Assumption A1 ("LLM-generated code in typed Python has extractable static analysis constraints") was the core hypothesis of h-e1 and remains unverified. If LLM code quality is insufficient—incomplete type annotations, inconsistent Pydantic validator usage, malformed AST structures—the entire incremental SMT approach fails at the foundation. Unlike infrastructure failures (fixable), poor LLM code quality would be a fundamental limitation requiring either LLM fine-tuning, post-processing repair, or pivot to neural constraint extraction. This condition was enforced via prompts but never tested on actual LLM output.

Limitation 3: No Baseline Comparison—Speedup Magnitude Unverified

The hypothesis predicted speedup vs batch SMT verification. No experiments measured wall-clock verification time. Without empirical speedup data, it cannot be determined if the approach provides practical benefit over batch verification. Even if constraint extraction works (h-e1), incremental SMT might only provide marginal benefit rather than the predicted speedup. Future resolution requires completing the full hypothesis chain.

Limitation 4: Single Language Scope—Generalization Uncertain

Experiment design focused exclusively on typed Python with Pydantic. Generalization to Rust was not tested. Claims about "statically-typed languages" (plural) are overstated; only one language configuration (Python + Pydantic + Pyre) was designed for testing. Transferability to Rust depends on tool availability (Prusti), type system differences (borrow checker, ownership semantics), and LLM training data distribution. Results cannot generalize beyond Python + Pydantic without additional experiments.

Limitation 5: Repair Locality Assumption (A2) May Not Hold for LLM Code

Assumption A2 ("LLM repairs are localized like human repairs") transfers from CURE/CoCoNut literature (80% of human repairs touch 1–3 lines) without validation. LLM repair patterns may differ significantly: broader edits, function regeneration, cross-module dependencies. If violated, dependency cones expand, reducing or eliminating speedup benefits even if extraction succeeds. Without validating A2 on LLM code, the speedup magnitude is highly uncertain. This assumption is as foundational as A1 but receives less attention—failure here would invalidate the incremental SMT approach even if static analysis works perfectly.

6.0.3. IMPLICATIONS FOR HYPOTHESIS PLAUSIBILITY

While the immediate failure is infrastructure (invalid API key), it must be considered whether this masks fundamental problems that would invalidate the hypothesis even after fixing authentication.

LLM Code Quality May Be Incompatible with Static Analysis: Static analyzers require well-formed AST, consistent type annotations, and absence of dynamic features. LLMs may generate incomplete annotations, inconsistent Pydantic usage, dynamic features, or unusual idioms valid in Python but unhandled by Pyre. If A1 fails (extraction <90%), this would indicate fundamental incompatibility between LLM code generation and static analysis tooling. The pivot would require either LLM fine-tuning on typed code with static analyzer validation in training loop, or

neural constraint extraction replacing Pyre entirely.

Static Analyzer Brittleness on Neural Code: Pyre is validated on human-written code with IDE feedback and developer review cycles. LLM code lacks this correction loop. Even if LLMs produce syntactically valid typed Python, Pyre may fail on edge cases in type inference, annotation style variation, or validator complexity.

Repair Locality Assumption May Be Violated: Human developers make surgical edits. LLMs may behave differently: function regeneration, cascading changes, cross-module dependencies. If A2 fails (average cone size >50%), incremental SMT provides minimal speedup over batch. Even if extraction works (h-e1 passes), the approach becomes impractical.

For the hypothesis to remain viable after retry, h-e1 must pass strongly (extraction $\geq 90\%$), extracted constraints must be meaningful (actual preconditions/postconditions mappable to SMT predicates, not just trivial type annotations), and heterogeneity in failures must be low.

Current Evidence: Homogeneous failure mode (100% API auth errors, zero heterogeneous extraction/parsing failures) suggests infrastructure blockage rather than code quality issues. However, this is weak evidence—absence of observed failures is not evidence of capability. The hypothesis remains in a "Schrödinger's cat" state: potentially sound OR fundamentally flawed, unknowable until h-e1 executes.

6.0.4. LESSONS LEARNED

Lesson 1: Pre-flight validation is critical for experiments with external API dependencies. A single test API call before starting a 100-request batch experiment would have detected the authentication error immediately, saving wasted execution time.

Lesson 2: Modular pipeline architecture enables partial validation despite dependency failures. Separating concerns (dataset preparation, model execution, evaluation) allowed dataset extension to succeed independently of LLM generation failure.

Lesson 3: Negative results from infrastructure failures are publishable when methodology is sound. Scientific integrity requires distinguishing between hypothesis refutation and experimental blockage. The hypothesis remains plausible, the implementation is complete, and the dataset artifact is reusable.

6.0.5. FUTURE WORK RECOMMENDATIONS

Immediate: Retry h-e1 with valid Anthropic API key. If `extraction_rate` $\geq 90\%$, proceed to h-m1-m3 validation. If `extraction_rate` $< 90\%$, pivot to neural constraint extraction.

Medium-term: Extend dataset to Rust + Prusti for language generalization. Test whether incremental SMT transfers beyond Python. Validate repair locality assumption (A2) on LLM code.

Long-term: Standardize typed benchmarks for formal verification research across multiple languages (Python + Pydantic, Rust + Prusti, TypeScript + type predicates). Establish evaluation protocol for comparing neural code generation with/without SMT verification.

7. Conclusion

This paper reports on a validation attempt for the hypothesis that incremental SMT verification would reduce verification time over batch re-verification for LLM code repair in typed Python with Pydantic. The validation pipeline failed before generating a single line of code due to API authentication errors, revealing that infrastructure robustness is a prerequisite to hypothesis testing.

The hypothesis—that incremental SMT verification (re-verifying only modified functions and dependencies) reduces verification time compared to batch re-verification—remains theoretically plausible but empirically unvalidated. All experimental validation was blocked at the foundational hypothesis (h-e1: static analyzers can extract SMT constraints from LLM-generated typed code) due to 100% API authentication failure rate (48/48 generation attempts returned HTTP 401 "API key is invalid").

Despite generation failure, the dataset extension pipeline succeeded: HumanEval was mechanically extended with 100 Pydantic-annotated prompts. This artifact addresses a gap in formal verification research—prior benchmarks lack type annotations required for SMT constraint extraction. The constraint extraction pipeline (DatasetExtender $\rightarrow$ LLMGenerator $\rightarrow$ Pyre-Extractor $\rightarrow$ Z3Validator $\rightarrow$ MetricsEngine) was validated on error files, confirming architectural soundness on negative cases.

The contributions are:

1. **HumanEval + Pydantic Type Extensions:** 100 mechanically-generated type-annotated prompts suitable for SMT constraint extraction

research, addressing the gap in typed benchmarks for formal verification of LLM code.

1. **Constraint Extraction Pipeline Architecture:** A modular pipeline ready for retry, with components independently validated on error cases.

The primary methodological contribution is the demonstration that infrastructure reliability is a prerequisite to hypothesis testing. Experimental pipelines must address failure modes explicitly through pre-flight API key validation, fast-fail on auth errors, and checkpoint/resume for long-running experiments.

Honest Assessment of Claims:

- Cannot claim "incremental SMT achieves speedup" (untested)
- Cannot claim "speedup scales with codebase size" (untested)
- Cannot claim "conservative dependency analysis maintains soundness" (untested)
- Cannot claim "type annotations enable constraint extraction from LLM code" (untested)

All quantitative predictions await validation pending infrastructure fixes and completion of the $h-e1 \rightarrow h-m1 \rightarrow h-m2 \rightarrow h-m3$ hypothesis chain.

Limitations: Single language scope (only Python + Pydantic + Pyre tested). Untested assumptions (A1: LLM code has extractable constraints; A2: repair locality transfers from human to LLM code). No baseline comparison (speedup figure is speculative, not measured).

Future Work: Retry $h-e1$ with valid Anthropic API key. If `extraction_rate` $\geq 90\%$, proceed to mechanism testing. If `extraction_rate` $< 90\%$, pivot to neural constraint extraction. Extend approach to Rust + Prusti for language generalization. Validate repair locality assumption on LLM code. Standardize typed benchmarks for formal verification research across multiple languages.

The HumanEval + Pydantic dataset extension is the artifact from this validation attempt—a resource for future constraint extraction research, and a reminder that experimental methodology extends beyond hypothesis design to encompass infrastructure reliability.

References

- Astrauskas, V., Müller, P., Poli, F., and Summers, A. J. Leveraging rust types for modular specification and verification. In *OOPSLA*, 2019.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code, 2021.
- Colvin, S. et al. Pydantic: Data validation and settings management using python type annotations. 2020.
- de Moura, L. and Bjørner, N. Z3: An efficient smt solver. In *TACAS*, 2008.
- Facebook. Pyre: Fast type checking for python, 2018.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., et al. Measuring coding challenge competence with apps, 2021.
- Jiang, N., Lutellier, T., and Tan, L. Cure: Code-aware neural machine repair. In *ICSE-SEIP*, 2019.
- Li, Y., Choi, D., Chung, J., Kushman, N., et al. Competition-level code generation with alphacode. In *Science*, 2022.
- Long, F. and Rinard, M. Automated patch synthesis for human-written programs via symbolic analysis. In *FSE*, 2016.
- Lutellier, T., Pham, H. V., Pang, L., Li, Y., Wei, M., and Tan, L. Coconut: Combining context-aware neural translation models using ensemble for program repair. In *ISSTA*, 2020.
- Mechtaev, S., Yi, J., and Roychoudhury, A. Angelix: Scalable multiline program patch synthesis via symbolic analysis. In *ICSE*, 2016.
- Various. Code generation benchmarks: Codeforces and beyond. 2022.
- Wang, Y., Wang, W., Joty, S., and Hoi, S. C. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, 2021.